

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Обход графа в ширину"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М.А.

Принял:

к.т.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение обхода графа в ширину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

Псевдокод

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОШ

- 1.1. для всех i положим $NUM[i] = \text{False}$ пометим как "не посещенную";
- 1.2. **ПОКА** существует "новая" вершина v
- 1.3. **ВЫПОЛНЯТЬ** BFS (v).

Алгоритм BFS(v):

- 2.1. Создать пустую очередь $Q = \{\}$;
- 2.2. Поместить v в очередь $Q.push(v)$;
- 2.3. пометить v как "посещенную" $NUM[v] = \text{True}$;
- 2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**
- 2.5. $v = Q.front()$ установить текущую вершину;
- 2.6. Удалить первый элемент из очереди $Q.pop()$;
- 2.7. вывести на экран v ;
- 2.8. **ДЛЯ** $i = 1$ **ДО** $size_G$ **ВЫПОЛНЯТЬ**
- 2.9. **ЕСЛИ** $G(v,i) = 1$ **И** $NUM[i] = \text{False}$
- 2.10. **ТО**

2.11. Поместить i в очередь $Q.push(i)$;

2.12. пометить v как "посещенную" $NUM[v] = True$;

Реализация состоит из подготовительной части, в которой все вершины помечаются как не посещенные (п.1.1) и осуществляется запуск процедуры обхода для вершин графа (п.1.2, 1.3).

В самой процедуре обхода сначала создается пустая очередь (п. 2.1), в которую помещается исходная вершина, из которой начат обход (п.2.2).

Далее итерационно, пока очередь не опустеет, из нее извлекается первый элемент, который становится текущей вершиной (п. 2.5, 2.6). Затем в цикле просматривается v -я строка матрицы смежности графа $G(v,i)$. Как только алгоритм встречает смежную с v не посещенную вершину (п.2.9), эта вершина помещается в очередь (п.2.11) и помечается как посещенная (п.2.12). После просмотра строки матрицы смежности алгоритм делает следующую итерацию цикла 2.4 или заканчивает работу, если очередь пуста.

Например, если для графа на рисунке 1 начать обход из первой вершины, то вершины 3, 6 и 2 будут находиться на уровне удаленности в 1 ребро, а вершины 5 и 4 на уровне удаленности в 2 ребра.

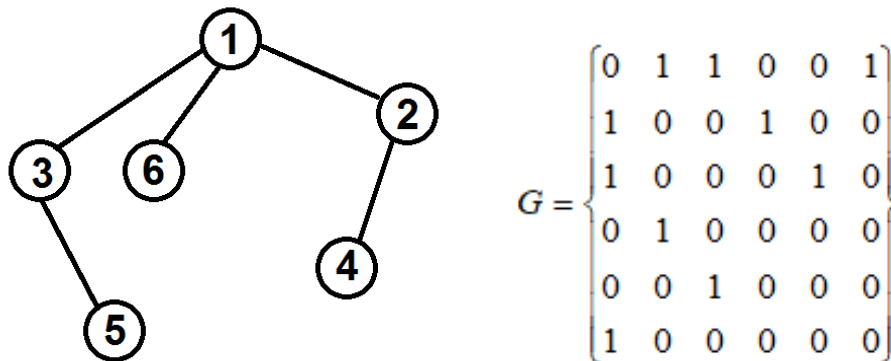


Рисунок 1 – Граф

Тогда при обходе этого графа в ширину, мы сначала посетим вершины первого уровня удаленности (с номерами 2, 3 и 6), и только после того, как закончатся не посещенные вершины на этом уровне, мы перейдем к следующему. На втором уровне мы посетим все вершины, которые удалены от исходной на 2 ребра (вершины 4 и 5).

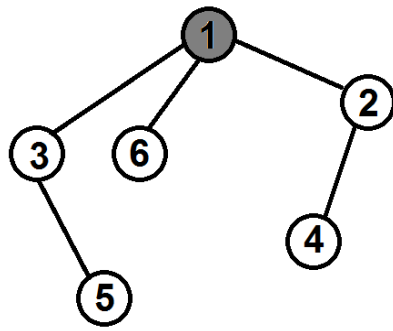


Рисунок 2 – Итерация 1

При просмотре 1-й строки матрицы смежности

$$G = \begin{Bmatrix} 0 & \boxed{1} & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

будет найдена смежная вершина с индексом 2 ($G(1,2) = 1$), которая не посещена ($NUM[2] = \text{False}$) и она будет помещена в очередь и помечена ($NUM[2] = \text{True}$). Просмотр строки продолжится, и в очередь будут также помещены и помечены как посещенные вершины с индексами 3 и 6.

К концу первой итерации очередь будет содержать $Q = \{2, 3, 6\}$.

Условие цикла *while* будет выполнено и на следующей итерации вершина 2 будет установлена как текущая и извлечена из очереди, на экран будет выведена двойка.

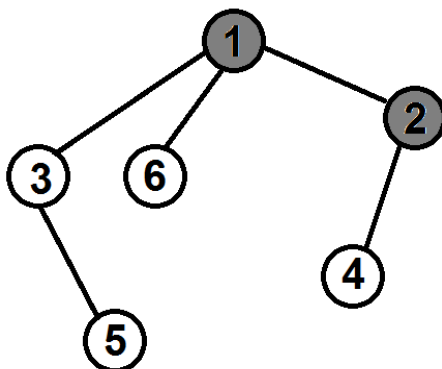


Рисунок 3 – Итерация 2

И алгоритм перейдет к просмотру второй строки матрицы смежности.

Первая смежная с вершиной 2 - вершина с индексом 1 ($G(2,1) = 1$),

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

которая к настоящему моменту уже посещена ($NUM[1] = \text{True}$), она не будет помещена в очередь. Цикл 2.9 продолжит просмотр матрицы смежности.

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Следующая найденная вершина, смежная со второй, будет иметь индекс 4 ($G(2,4) = 1$), она не посещена ($NUM[4] = \text{False}$) и она будет помещена в очередь и помечена.

К концу второй итерации очередь будет содержать $Q = \{3, 6, 4\}$.

Условие цикла *while* будет выполнено и на следующей итерации вершина 3 будет установлена как текущая и извлечена из очереди, на экран будет выведена тройка.

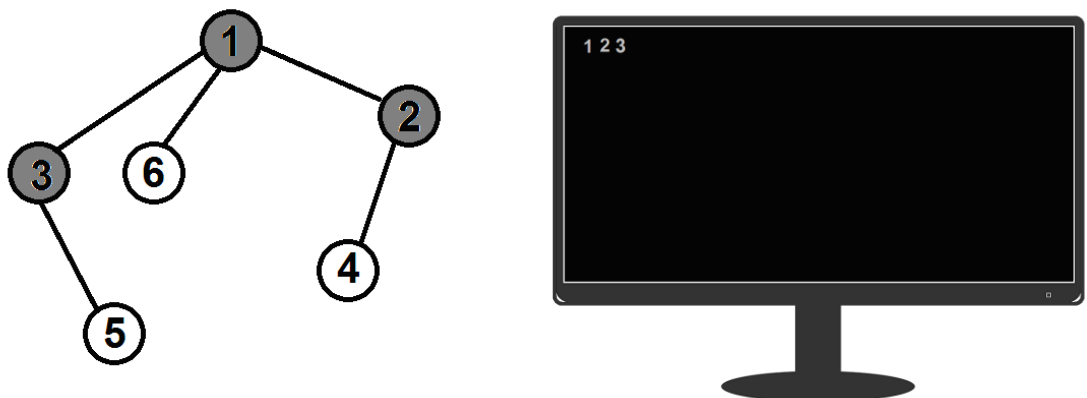


Рисунок 4 – Итерация 3

При просмотре 3-й строки матрицы будет найдена вершина 1, но она уже посещена ($NUM[1] = \text{True}$), поэтому в очередь помещена не будет.

$$G = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Следующая найденная вершина, смежная с третьей – вершина с номером 5 ($G(3,5) = 1$), она не посещена ($NUM[5] = \text{False}$) и будет помещена в очередь.

$$G = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

К концу третьей итерации очередь будет содержать $Q = \{6, 4, 5\}$.

Условие цикла *while* будет выполнено. На следующей итерации вершина 6 будет установлена как текущая и извлечена из очереди, на экран будет выведена цифра шесть.

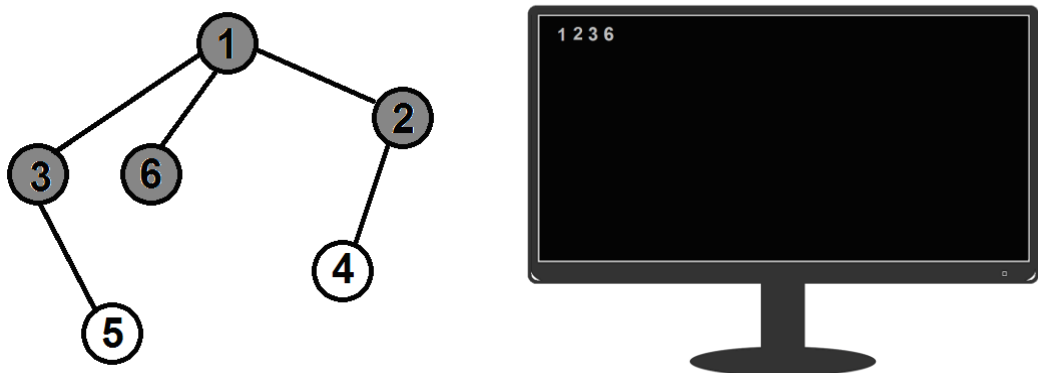


Рисунок 5 – Итерация 4

При просмотре 6-й строки матрицы будет найдена вершина 1, но она уже посещена ($NUM[1] = \text{True}$), поэтому в очередь помещена не будет.

К концу четвертой итерации очередь будет содержать $Q = \{4, 5\}$.

Далее из очереди будет извлечена вершина 4, установлена как текущая и

выведена на экран.

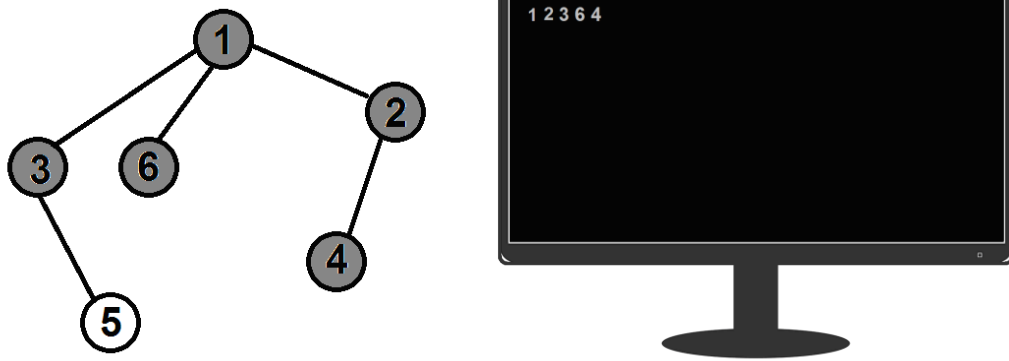


Рисунок 6 – Итерация 5

Так как все вершины кроме пятой уже посещены, то при просмотре 4-й строки в матрице смежности в очередь не будут добавлены вершины.

К концу пятой итерации очередь будет содержать только одну вершину $Q = \{5\}$.

На шестой итерации вершина с номером пять будет установлена как текущая и извлечена из очереди, на экран будет выведена цифра пять.

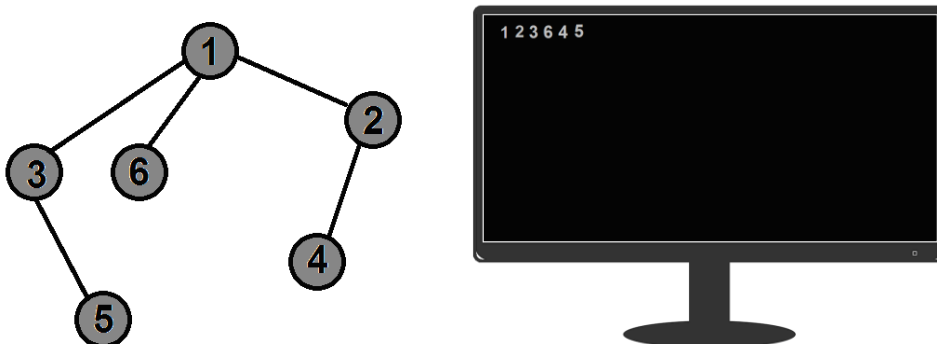


Рисунок 7 – Результат работы обхода

Просмотр 5-й строки матрицы смежности не добавит в очередь новых вершин, так как к этому моменту они уже все помечены как посещенные.

После этой итерации очередь окажется пуста $Q = \{\}$ и алгоритм завершит свою работу.

В конце работы алгоритма все вершины будут посещены. А на экран будут выведены номера вершин в порядке их посещения алгоритмом.

Результат программы

```
E:\gitnew\laba8\Debug\laba8.exe
Input number of vertission : 6
0 1 1 0 0 1
1 0 0 0 0 0
1 0 0 0 1 1
0 0 0 0 1 1
0 0 1 1 0 1
1 0 1 1 1 0
Input start vert : 4
Path: 4 2 3 5 0 1
```

Рисунок 8 – Обход графа в ширину

```
E:\gitnew\laba8\Debug\laba8.exe
Input number of vertission : 10
0 1 1 0 0 1 0 0 0 0
1 0 0 1 1 1 1 1 1 1
1 0 0 0 1 0 1 0 0 1
0 1 0 0 0 0 1 0 0 1
0 1 1 0 0 1 0 1 0 1
1 1 0 0 1 0 0 1 1 1
0 1 1 1 0 0 0 0 1 1
0 1 0 0 1 1 0 0 0 1
0 1 0 0 0 1 1 0 0 1
0 1 1 1 1 1 1 1 1 0
Input start vert : 5
Path: 5 0 1 4 7 8 9 2 3 6
```

Рисунок 9 – Обход графа в ширину

Листинг

Файл 1- laba8.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <stdio.h>
#include <conio.h>
#include <cstdlib>
#include <ctime>
#include <queue>

using namespace std;

void BFS(int** G, int numG, int* visited, int s) {
    queue<int> q;
    int v;
    visited[s] = 1;
    q.push(s);

    while (!q.empty()) {
        v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && visited[i] == 0) {
                q.push(i);
                visited[i] = 1;
            }
        }
    }
}

int main() {
    int** G;
    int* visited;
    int numG, current;

    printf("Input number of vertission : ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++){
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for(int i = 0; i < numG; i++){
        visited[i] = 0;
        for (int j = i; j < numG; j++){
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for(int i = 0; i < numG; i++){
        for (int j = 0; j < numG; j++){
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }

    printf("Input start vert : ");
    scanf("%d", &current);
```

```
printf("Path: ");

BFS(G, numG, visited, current);
printf("\n\n");

for (int i = 0; i < numG; i++)
    free(G[i]);
free(G);
free(visited);

getchar();
getchar();

return 0;
}
```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №8. В процессе выполнения работы были использованы знания обхода графов в ширину.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>